PARIS PANTHÉON-ASSAS UNIVERSITÉ

Report for Graph Theory Project : Floyd warshall algorithm

Ewenn BAUDET, Félix LANG, Daïf MOINDJIE, Timothée TACCA et Inès VEIGNEAU

Table of contents:

Real life example

1. Context of the problem

On a Monday morning in France, a carpooling application connects several cities through available rides. Each ride has a specific travel time, and not all cities are directly connected. Therefore, travelers may need to go through intermediate cities to reach their destination.

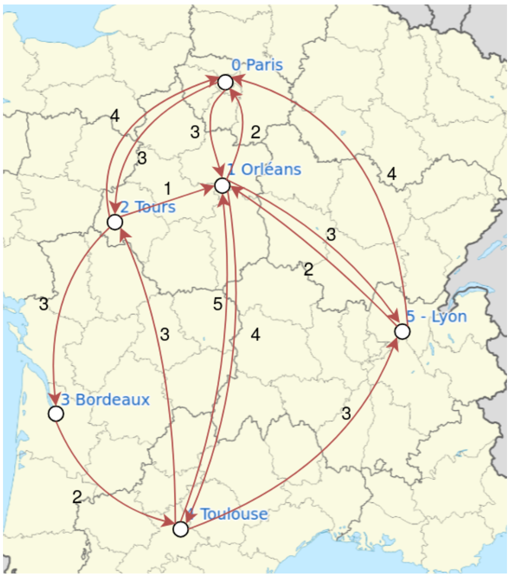
The platform considers the following five cities:
0: Paris
1: Orléans
2: Tours
3: Bordeaux
4: Toulouse
5: Lyon

Each available carpooling route is modeled as a directed edge in a graph, where:
- Vertices represent cities,
- Edges represent available rides,
- Weights represent the travel time in hours.

2. Available data

There are 14 available travels as follows:
- Paris → Orléans: 3 hours
- Paris → Tours: 3 hours
- Orléans → Paris: 2 hours
- Orléans → Toulouse: 4 hours
- Orléans → Lyon: 3 hours
- Tours → Orléans: 1 hours
- Tours → Bordeaux: 3 hours
- Tours → Paris: 4 hours
- Bordeaux → Toulouse: 2 hours
- Toulouse → Tours: 3 hours
- Toulouse → Orléans: 5 hours
- Toulouse → Lyon: 3 hours
- Lyon → Orléans: 2 hours
- Lyon → Paris: 4 hours

We can model this problem with the following graph :



3. Floyd-Warshall algorithm application

We want to determine the shortest travel time between every pair of cities. In order to find this shortest path we will apply the Floyd-Warshall algorithm. This algorithm systematically improves the shortest paths by considering each city as an intermediate node.

We can model our graph as the following file:

```
6
14
0 1 3
0 2 3
1 0 2
1 4 4
1 5 3
2 0 4
2 1 1
2 3 3
3 4 2
4 1 5
4 2 3
4 5 3
5 0 4
5 1 2
```

After applying the algorithm, we obtain the following distance matrices :

```
Adjacency Matrix:
     0  1  2  3  4  5
  0  0  3  3  6  7  6
  1  2  0  5  8  4  3
  2  3  1  0  3  5  4
  3  8  6  5  0  2  5
  4  6  4  3  6  0  3
  5  4  2  7 10  6  0

Predecessor Matrix:
     0  1  2  3  4  5
  0  _  0  0  2  1  1
  1  1  _  0  2  1  1
  2  1  2  _  2  1  1
  3  1  2  4  _  3  4
  4  1  2  4  2  _  4
  5  5  5  0  2  1  _
```

4. <u>Interpretation of the obtained results</u>

Let’s assume we want to find the shortest path to go from Toulouse (*which is represented by node 4*) and Paris (which is represented by node 0)

The adjacency matrix contains the cost of the shortest path between two nodes of the graph. Therefore, looking at the above adjacency matrix, we can already know that the total cost of said path will be 6. (*line 4, column 0*).

In order to actually determine the full path, we now need to look at the predecessor Matrix. Because the destination is Paris, we first look at column 0. At row 4 *(which represents Toulouse*) of that column, there is a 1 (*which represents Orléan*s). It means the predecessor of Paris in the shortest path between Toulouse and Paris is Orléans.

We can then repeat this process multiple times (*by each time considering the predecessor to be the destination*) until we reach the source. If we look at column 1 row 4 there is a 2 (*which represents Tours*).

Finally, looking at column 2 row 4, there is a 4 which corresponds to Toulouse, our starting point.

From this, we can therefore conclude that the shortest path between Toulouse and Paris is Toulouse→Tours →Orléans →Paris